Competitive Programming

Course Code: BCS24556B

Question Bank - Python Solutions, Line-by-Line Explanations and Dry Runs

Global Academy of Technology - Department of Computer Science and Engineering

Contents

Session 1: Data Types, Operators & Control Flow	Questions 1.1, 1.2
Session 2: Functions, Recursion & Memory Management	Questions 2.1, 2.2
Session 3: Pointers, Arrays & Strings	Questions 3.1, 3.2
Session 4: Structures, Unions & File Handling	Questions 4.1, 4.2
Session 5: Linked Lists in C	Questions 5.1, 5.2
Session 6: STL - Stacks, Queues & Priority Queues	Questions 6.1, 6.2
Session 7: Linked Lists, Binary Trees, Stacks & Queues	Questions 7.1, 7.2
Session 8: String Matching	Questions 8.1, 8.2
Session 9: Performance Analysis of Sorting and Searching	Questions 9.1, 9.2
Session 10: Number Theory	Questions 10.1, 10.2

Each question has: 1. Problem | 2. Python solution | 3. Line-by-line explanation | 4. Dry run

Every program was run against the sample test case from the question bank and gives the same output. Python has no pointers, malloc or unions, so questions written for C use the closest Python equivalent: lists for dynamic memory, a class for the tagged union, and the struct module for the binary file.

Session 1: Data Types, Operators & Control Flow

Question 1.1

Single Unique ID Finder (Bitwise Operations)

Marks: 10 | Bloom's Taxonomy Level: L3 (Apply)

1 Problem

In a smart warehouse, inventory tags are tracked using 32-bit integer IDs. Due to a system glitch, every tag ID in a log file appears exactly twice, except for one unique tag ID which appears only once. Write a C program to identify the unique tag ID.

Constraints / Requirements

- Your solution must run in O(N) time complexity and use O(1) auxiliary space.
- You are strictly required to use Bitwise XOR (^) operations without auxiliary arrays or hashing.

Input Format

First line contains an integer N (1 <= N <= 10^5), representing the number of elements in the array (always odd).
Second line contains N space-separated integers representing the tag IDs.

Output Format

Print the single integer that appears only once.

Sample Input	Sample Output
5 4 1 2 1 2	4

2 Python Solution

```
1  import sys
2
3  data = sys.stdin.read().split()
4  n = int(data[0])
5  result = 0
6  for i in range(1, n + 1):
7      result ^= int(data[i])
8  print(result)
```

3 Line-by-Line Explanation

Line	Code	Explanation
1	`import sys`	Import the sys module so we can read all the input at once.
3	`data = sys.stdin.read().split()`	Read the whole input and split it into a list of text tokens.
4	`n = int(data[0])`	The first token is N, the number of IDs.
5	`result = 0`	Start from 0, because x ^ 0 = x.
6	`for i in range(1, n + 1):`	Visit tokens 1 to N, which are the tag IDs.
7	`result ^= int(data[i])`	XOR each ID into result. Equal IDs cancel out (a ^ a = 0), so only the unique ID remains.
8	`print(result)`	Print the unique ID.

4 Dry Run (Sample Test Case)

Input: N = 5 and IDs = 4 1 2 1 2. result starts at 0. In binary: 4 = 100, 1 = 001, 2 = 010.

i	data[i]	result before	result ^ data[i] (binary)	result after
1	4	0	000 ^ 100 = 100	4
2	1	4	100 ^ 001 = 101	5
3	2	5	101 ^ 010 = 111	7
4	1	7	111 ^ 001 = 110	6
5	2	6	110 ^ 010 = 100	4

Result

```
The pairs (1, 1) and (2, 2) cancelled out, leaving 4. Output: 4
```

Question 1.2

32-Bit Integer Reversal with Overflow Guard

Marks: 10 | Bloom's Taxonomy Level: L3 (Apply)

1 Problem

Given a signed 32-bit integer N, write a C function int reverseInteger(int n) that reverses its digits using basic arithmetic operations (% and /) and iterative loops.
If reversing N causes the value to overflow or underflow the 32-bit signed integer range [-2^31, 2^31 - 1], the function must return 0.

Input Format

A single signed 32-bit integer N.

Output Format

A single integer representing the reversed value, or 0 if an overflow occurs.

Sample Input	Sample Output
-1230	-321

2 Python Solution

```
def reverseInteger(n):
    sign = -1 if n < 0 else 1
    n = abs(n)
    rev = 0
    while n > 0:
        rev = rev * 10 + n % 10
        n = n // 10
    rev = rev * sign
    if rev < -2**31 or rev > 2**31 - 1:
        return 0
    return rev


n = int(input())
print(reverseInteger(n))
```

3 Line-by-Line Explanation

Line	Code	Explanation
1	`def reverseInteger(n):`	Define the function that reverses the digits of n.
2	`sign = -1 if n < 0 else 1`	Remember the sign: -1 for negative numbers, otherwise 1.
3	`n = abs(n)`	Work with the positive value so % and // are simple.
4	`rev = 0`	rev will hold the reversed number as we build it.
5	`while n > 0:`	Repeat until all digits have been used.
6	`rev = rev * 10 + n % 10`	Shift rev one place left and add the last digit of n.
7	`n = n // 10`	Remove the last digit from n.
8	`rev = rev * sign`	Put the original sign back.
9	`if rev < -2**31 or rev > 2**31 - 1:`	Check whether the answer is outside the 32-bit signed range.
10	`return 0`	Overflow or underflow, so return 0 as required.
11	`return rev`	Otherwise return the reversed number.

Line	Code	Explanation
14	`n = int(input())`	Read the integer N.
15	`print(reverseInteger(n))`	Call the function and print the answer.

4 Dry Run (Sample Test Case)

Input: n = -1230. First sign = -1 and n becomes 1230. rev starts at 0.

Loop	n (start)	last digit n % 10	rev = rev*10 + digit	n after n // 10
1	1230	0	0*10 + 0 = 0	123
2	123	3	0*10 + 3 = 3	12
3	12	2	3*10 + 2 = 32	1
4	1	1	32*10 + 1 = 321	0

Result

The loop stops at n = 0. rev = 321, then rev * sign = -321. This is inside the 32-bit range, so it is returned. Output: -321

Overflow example: n = 1534236469 reverses to 9646324351, which is more than 2147483647, so the function returns 0.

Session 2: Functions, Recursion & Memory Management

Question 2.1

Optimized Tail-Recursive Exponentiation with Callback Validation

Marks: 10 | Bloom's Taxonomy Level: L3 (Apply)

1 Problem

Implement a fast exponentiation function in C to compute x^n in O(log n) time using Tail Recursion.
Your solution must fulfill the following:
1. Define a function signature double power_tail(double x, long long n, double accumulator).
2. Define a callback function pointer typedef int (*validator_fn)(double, long long) that validates edge cases.
3. Write a wrapper function double my_pow(double x, long long n, validator_fn validate) that calls the tail-recursive function.

Input Format

A double value x representing the base and a signed 64-bit integer n representing the exponent separated by space.

Output Format

Print the result formatted to 5 decimal places. If an invalid math condition occurs (e.g., 0^-1), print INVALID.

Sample Input	Sample Output
2.00000 -10	0.00098

2 Python Solution

```
def power_tail(x, n, accumulator):
    if n == 0:
        return accumulator
    if n % 2 == 1:
        accumulator = accumulator * x
    return power_tail(x * x, n // 2, accumulator)


def validator_fn(x, n):
    # returns False for invalid cases such as 0^-1
    if x == 0 and n < 0:
        return False
    return True


def my_pow(x, n, validate):
    if not validate(x, n):
        return "INVALID"
    if n < 0:
        x = 1 / x
        n = -n
    result = power_tail(x, n, 1.0)
    return "%.5f" % result


parts = input().split()
x = float(parts[0])
n = int(parts[1])
print(my_pow(x, n, validator_fn))
```

3 Line-by-Line Explanation

Line	Code	Explanation
1	`def power_tail(x, n, accumulator):`	Tail-recursive fast power. The accumulator carries the answer built so far.
2	`    if n == 0:`	Base case: when the exponent reaches 0, the accumulator is the answer.
3	`        return accumulator`	Return the accumulated result.
4	`    if n % 2 == 1:`	If the exponent is odd, one extra factor of x must be multiplied in.
5	`        accumulator = accumulator * x`	Multiply the accumulator by the current base.
6	`    return power_tail(x * x, n // 2, accumulator)`	Square the base and halve the exponent. This call is the last action (tail call), so only O(log n) calls are made.
9	`def validator_fn(x, n):`	Callback function that checks for invalid math cases.
10	`    # returns False for invalid cases such as 0^-1`	Comment: explains what the validator does.
11	`    if x == 0 and n < 0:`	0 raised to a negative power would mean dividing by zero.
12	`        return False`	Tell the caller the input is invalid.
13	`    return True`	Every other input is valid.
16	`def my_pow(x, n, validate):`	Wrapper function. It receives the validator function as a parameter (the callback).
17	`    if not validate(x, n):`	Call the callback. If it reports invalid input...
18	`        return "INVALID"`	...return the word INVALID.
19	`    if n < 0:`	For a negative exponent, x^(-n) = (1/x)^n.
20	`        x = 1 / x`	Replace the base by its reciprocal.
21	`        n = -n`	Make the exponent positive.
22	`    result = power_tail(x, n, 1.0)`	Start the recursion with accumulator = 1.0.
23	`    return "%.5f" % result`	Format the answer to 5 decimal places.
26	`parts = input().split()`	Read the line and split it into the base and the exponent.
27	`x = float(parts[0])`	Convert the base to a decimal number.
28	`n = int(parts[1])`	Convert the exponent to an integer.
29	`print(my_pow(x, n, validator_fn))`	Pass validator_fn as the callback and print the result.

4 Dry Run (Sample Test Case)

Input: x = 2.0, n = -10. validator_fn(2.0, -10) returns True (x is not 0). Since n < 0, x becomes 1/2 = 0.5 and n becomes 10. Recursion starts with accumulator = 1.0.

Call	x	n	n odd?	accumulator after	Next call
1	0.5	10	No	1.0	power_tail(0.25, 5, 1.0)
2	0.25	5	Yes	1.0 * 0.25 = 0.25	power_tail(0.0625, 2, 0.25)
3	0.0625	2	No	0.25	power_tail(0.00390625, 1, 0.25)
4	0.00390625	1	Yes	0.25 * 0.00390625 = 0.0009765625	power_tail(x*x, 0, 0.0009765625)
5	(tiny)	0	-	0.0009765625	Base case: return accumulator

Result

Result = 0.0009765625, formatted with %.5f gives 0.00098. Output: 0.00098

```
Invalid example: input 0 -1 makes validator_fn return False, so the program prints
INVALID.
```

Question 2.2

Recursive Subset Generation via Pass-by-Reference

Marks: 10 | Bloom's Taxonomy Level: L3 (Apply)

1 Problem

Given a set of N distinct positive integers, write a recursive C program to generate and print all possible unique subsets (the power set).

Constraints / Requirements

- Memory allocation for storing current subset paths must be handled dynamically and passed down the call stack using Pass-by-Reference (pointers).
- Do not use global variables.

Input Format

First line: Integer N (1 <= N <= 10).
Second line: N space-separated distinct integers.

Output Format

Print each subset on a new line in lexicographical format. Print [] for the empty set.

Sample Input	Sample Output
3 1 2 3	[] [1] [1, 2] [1, 2, 3] [1, 3] [2] [2, 3] [3]

2 Python Solution

```
 1  def generate(arr, start, current):
 2      print(current)
 3      for i in range(start, len(arr)):
 4          current.append(arr[i])
 5          generate(arr, i + 1, current)
 6          current.pop()
 7
 8
 9  n = int(input())
10  arr = list(map(int, input().split()))
11  arr.sort()
12  generate(arr, 0, [])
```

3 Line-by-Line Explanation

Line	Code	Explanation
1	`def generate(arr, start, current):`	Recursive function. current is a list passed by reference, so every call shares the same list.
2	`print(current)`	Each call prints exactly one subset: the current list.
3	`for i in range(start, len(arr)):`	Try every element from position start onwards.
4	`current.append(arr[i])`	Choose arr[i]: add it to the subset.
5	`generate(arr, i + 1, current)`	Recurse to extend the subset using only the later elements.

Line	Code	Explanation
6	`current.pop()`	Backtrack: remove arr[i] so the next choice starts clean.
9	`n = int(input())`	Read N, the number of elements.
10	`arr = list(map(int, input().split()))`	Read the N integers into a list.
11	`arr.sort()`	Sort the list so subsets are printed in lexicographical order.
12	`generate(arr, 0, [])`	Start with an empty subset and index 0.

4 Dry Run (Sample Test Case)

Input: arr = [1, 2, 3]. Every call to generate() prints the current list first, then tries to add each later element one by one (choose, recurse, undo).

#	Call generate(start, current)	Printed	What happens next
1	generate(0, [])	[]	choose 1, recurse
2	generate(1, [1])	[1]	choose 2, recurse
3	generate(2, [1, 2])	[1, 2]	choose 3, recurse
4	generate(3, [1, 2, 3])	[1, 2, 3]	loop is empty; return; pop 3, pop 2; now choose 3 after [1]
5	generate(3, [1, 3])	[1, 3]	return; pop 3, pop 1; now choose 2 after []
6	generate(2, [2])	[2]	choose 3, recurse
7	generate(3, [2, 3])	[2, 3]	return; pop 3, pop 2; now choose 3 after []
8	generate(3, [3])	[3]	return; all loops finish

Result

Eight lines are printed in the order shown above (2^3 = 8 subsets), which is the required output.

Session 3: Pointers, Arrays & Strings

Question 3.1

In-Place 2D Matrix Rotation using Pointer Arithmetic

Marks: 10 | Bloom's Taxonomy Level: L3 (Apply)

1 Problem

Write a C program that dynamically allocates a 2D N x N matrix of integers using double pointers (int**). The program must rotate the matrix 90 degrees clockwise in-place using pure pointer arithmetic (avoiding array subscript notation matrix[i][j]).

Constraints / Requirements

- Dynamic allocation using malloc/calloc.
- Perform rotation in O(1) extra space by first transposing using pointers and then reversing each row.
- Free all dynamically allocated memory before program termination.

Input Format

First line contains integer N (1 <= N <= 100).
Next N lines contain N space-separated integers each.

Output Format

Print the N x N rotated matrix.

Sample Input	Sample Output
3 1 2 3 4 5 6 7 8 9	7 4 1 8 5 2 9 6 3

2 Python Solution

```
import sys

data = sys.stdin.read().split()
n = int(data[0])
m = []
idx = 1
for i in range(n):
    row = []
    for j in range(n):
        row.append(int(data[idx]))
        idx += 1
    m.append(row)

# transpose
for i in range(n):
    for j in range(i + 1, n):
        m[i][j], m[j][i] = m[j][i], m[i][j]

# reverse each row
for i in range(n):
    l, r = 0, n - 1
    while l < r:
        m[i][l], m[i][r] = m[i][r], m[i][l]
        l += 1
        r -= 1

for i in range(n):
    print(" ".join(str(x) for x in m[i]))
```

3 Line-by-Line Explanation

Line	Code	Explanation
1	`import sys`	Used to read all the input at once.
3	`data = sys.stdin.read().split()`	Read every token of the input.
4	`n = int(data[0])`	The first token is N, the matrix size.
5	`m = []`	m will be the matrix: a list of rows.
6	`idx = 1`	idx points to the next unread token.
7	`for i in range(n):`	Build N rows.
8	`    row = []`	Start an empty row.
9	`    for j in range(n):`	Each row has N numbers.
10	`        row.append(int(data[idx]))`	Take the next number and add it to the row.
11	`        idx += 1`	Move to the next token.
12	`    m.append(row)`	Add the finished row to the matrix.
14	`# transpose`	Comment: step 1 is to transpose the matrix.
15	`for i in range(n):`	Go through each row i.
16	`    for j in range(i + 1, n):`	Only visit elements above the diagonal so each pair is swapped once.
17	`        m[i][j], m[j][i] = m[j][i], m[i][j]`	Swap the element with its mirror across the diagonal.
19	`# reverse each row`	Comment: step 2 is to reverse every row.
20	`for i in range(n):`	Go through each row.
21	`    l, r = 0, n - 1`	Two pointers: l at the left end, r at the right end.
22	`    while l < r:`	Continue until the pointers meet.
23	`        m[i][l], m[i][r] = m[i][r], m[i][l]`	Swap the two end elements.
24	`        l += 1`	Move the left pointer right.
25	`        r -= 1`	Move the right pointer left.
27	`for i in range(n):`	Print the rotated matrix row by row.
28	`    print(" ".join(str(x) for x in m[i]))`	Convert the numbers to text and join them with spaces.

4 Dry Run (Sample Test Case)

Input: N = 3 with the matrix below. Step 1 transposes it (swap m[i][j] with m[j][i] for j > i: pairs (0,1), (0,2), (1,2)). Step 2 reverses each row.

Stage	Row 0	Row 1	Row 2
Original	1 2 3	4 5 6	7 8 9
After swapping 2 and 4	1 4 3	2 5 6	7 8 9
After swapping 3 and 7	1 4 7	2 5 6	3 8 9
After swapping 6 and 8 (transpose done)	1 4 7	2 5 8	3 6 9
After reversing each row	7 4 1	8 5 2	9 6 3

Result

```
The matrix is rotated 90 degrees clockwise. Output:
7 4 1
8 5 2
9 6 3
```

Question 3.2

Custom String Compression Engine using Pointer Traversal

Marks: 10 | Bloom's Taxonomy Level: L3 (Apply)

1 Problem

Write a C function char* compressString(const char* str) that takes an input string and returns a dynamically allocated string representing its run-length compression (e.g., "aabcccccaaa" becomes "a2b1c5a3").

Constraints / Requirements

- You cannot use standard string library functions like strlen, strcpy, or strcat.
- All string traversals and manipulations must be executed using pointer incrementing and dereferencing.
- If the compressed string length is not smaller than the original string length, return a duplicate of the original string.

Input Format

A single string containing uppercase and lowercase ASCII letters (max length 1000).

Output Format

The compressed string (or original string if compression offers no length reduction).

Sample Input	Sample Output
aabcccccaaa	a2b1c5a3

2 Python Solution

```
 1  def length(x):
 2      count = 0
 3      for _ in x:
 4          count += 1
 5      return count
 6
 7
 8  def compressString(s):
 9      n = length(s)
10      result = ""
11      i = 0
12      while i < n:
13          j = i
14          while j < n and s[j] == s[i]:
15              j += 1
16          result += s[i] + str(j - i)
17          i = j
18      if length(result) < n:
19          return result
20      return s
21
22
23  s = input().strip()
24  print(compressString(s))
```

3 Line-by-Line Explanation

Line	Code	Explanation
1	`def length(x):`	Our own length function, because strlen is not allowed.
2	`count = 0`	Counter starts at 0.
3	`for _ in x:`	Walk over every character of x.

Line	Code	Explanation
4	`count += 1`	Count one more character.
5	`return count`	Return the total number of characters.
8	`def compressString(s):`	Function that returns the run-length compressed string.
9	`n = length(s)`	Length of the original string.
10	`result = ""`	The compressed text is built here.
11	`i = 0`	i marks the start of the current run of equal characters.
12	`while i < n:`	Repeat until the whole string is processed.
13	`j = i`	j will move to the end of the run.
14	`while j < n and s[j] == s[i]:`	Move j forward while the character stays the same.
15	`j += 1`	Extend the run by one character.
16	`result += s[i] + str(j - i)`	Append the character and its run length (j - i).
17	`i = j`	Start the next run where this one ended.
18	`if length(result) < n:`	Is the compressed string shorter than the original?
19	`return result`	Yes, so return the compressed string.
20	`return s`	No, so return the original string.
23	`s = input().strip()`	Read the string and remove extra spaces at the ends.
24	`print(compressString(s))`	Compress it and print the result.

4 Dry Run (Sample Test Case)

Input: aabccccaaa. Characters by index: 0 a, 1 a, 2 b, 3 c, 4 c, 5 c, 6 c, 7 c, 8 a, 9 a, 10 a. Original length n = 11.

Run	i (start)	j (end, exclusive)	Run length j - i	Appended	result so far
1	0	2	2	a2	a2
2	2	3	1	b1	a2b1
3	3	8	5	c5	a2b1c5
4	8	11	3	a3	a2b1c5a3

Result

`length(result) = 8, which is less than 11, so the compressed string is returned.`
`Output: a2b1c5a3`

Session 4: Structures, Unions & File Handling

Question 4.1

Student Registry Binary File Indexing & Filtering

Marks: 10 | Bloom's Taxonomy Level: L3 (Apply)

1 Problem

Design a C program for a university evaluation engine:
1. Define a structure Student with attributes: int id, char name[50], and float gpa.
2. Read data for N students and save the structures directly into a binary file named records.db using fwrite().
3. Open records.db, parse the records using fread(), dynamically allocate an array of Student structures for students with a GPA >= 8.5, and print their details.

Input Format

First line: Integer N.
Next N lines: id name gpa (space-separated).

Output Format

Print filtered records of eligible students (GPA >= 8.5), each on a new line formatted as: ID: <id> | Name: <name> | GPA: <gpa>.

Sample Input	Sample Output
3 101 Alice 9.1 102 Bob 7.8 103 Charlie 8.7	ID: 101 | Name: Alice | GPA: 9.10 ID: 103 | Name: Charlie | GPA: 8.70

2 Python Solution

```
import struct
import sys

FORMAT = "<i50sf"          # int id, char name[50], float gpa
SIZE = struct.calcsize(FORMAT)

data = sys.stdin.read().split()
n = int(data[0])

# write all records to the binary file
f = open("records.db", "wb")
idx = 1
for i in range(n):
    sid = int(data[idx])
    name = data[idx + 1].encode()
    gpa = float(data[idx + 2])
    idx += 3
    f.write(struct.pack(FORMAT, sid, name, gpa))
f.close()

# read them back and keep only GPA >= 8.5
eligible = []
f = open("records.db", "rb")
for i in range(n):
    chunk = f.read(SIZE)
    sid, name, gpa = struct.unpack(FORMAT, chunk)
    name = name.rstrip(b"\x00").decode()
    if gpa >= 8.5:
```

```
29          eligible.append((sid, name, gpa))
30  f.close()
31
32  for sid, name, gpa in eligible:
33      print("ID: %d | Name: %s | GPA: %.2f" % (sid, name, gpa))
```

3 Line-by-Line Explanation

Line	Code	Explanation
1	`import struct`	struct converts values to and from raw bytes, like fwrite/fread in C.
2	`import sys`	Used to read all the input at once.
4	`FORMAT = "<i50sf"  # int id, char name[50], float gpa`	Layout of one record: 4-byte int, 50-byte name, 4-byte float. The "<" means no padding.
5	`SIZE = struct.calcsize(FORMAT)`	Number of bytes in one record (58).
7	`data = sys.stdin.read().split()`	Read all input tokens.
8	`n = int(data[0])`	The first token is N, the number of students.
10	`# write all records to the binary file`	Comment: step 2, save the records.
11	`f = open("records.db", "wb")`	Open records.db in binary write mode.
12	`idx = 1`	idx points to the next unread token.
13	`for i in range(n):`	Repeat for each student.
14	`    sid = int(data[idx])`	The student ID.
15	`    name = data[idx + 1].encode()`	The name, converted to bytes so it can be stored.
16	`    gpa = float(data[idx + 2])`	The GPA as a decimal number.
17	`    idx += 3`	Move to the next student (3 tokens per student).
18	`    f.write(struct.pack(FORMAT, sid, name, gpa))`	Pack id, name and gpa into 58 bytes (name padded with zero bytes) and write them, like fwrite().
19	`f.close()`	Close the file so everything is saved.
21	`# read them back and keep only GPA >= 8.5`	Comment: step 3, read and filter.
22	`eligible = []`	Dynamic list to hold the students with GPA >= 8.5.
23	`f = open("records.db", "rb")`	Open the file again in binary read mode.
24	`for i in range(n):`	Read N records.
25	`    chunk = f.read(SIZE)`	Read exactly one record (58 bytes), like fread().
26	`    sid, name, gpa = struct.unpack(FORMAT, chunk)`	Unpack the bytes back into id, name and gpa.
27	`    name = name.rstrip(b"\x00").decode()`	Remove the zero padding and convert the bytes to text.
28	`    if gpa >= 8.5:`	Keep only students whose GPA is at least 8.5.
29	`        eligible.append((sid, name, gpa))`	Add this student to the list.
30	`f.close()`	Close the file.
32	`for sid, name, gpa in eligible:`	Print every eligible student.
33	`    print("ID: %d | Name: %s | GPA: %.2f" % (sid, name, gpa))`	Print in the required format; %.2f shows 2 decimals.

4 Dry Run (Sample Test Case)

Input: N = 3. Each record is packed into 58 bytes (4 + 50 + 4) and written to records.db. Then the file is read back one record at a time.

Student	Written to records.db	GPA >= 8.5 ?	Printed
101 Alice 9.1	58-byte record	9.1 >= 8.5, Yes	ID: 101 | Name: Alice | GPA: 9.10
102 Bob 7.8	58-byte record	7.8 >= 8.5, No	(skipped)
103 Charlie 8.7	58-byte record	8.7 >= 8.5, Yes	ID: 103 | Name: Charlie | GPA: 8.70

Result

```
Output:
ID: 101 | Name: Alice | GPA: 9.10
ID: 103 | Name: Charlie | GPA: 8.70
```

Question 4.2

Heterogeneous Sensor Log Parser using Tagged Unions

Marks: 10 | Bloom's Taxonomy Level: L3 (Apply)

1 Problem

An IoT device logs mixed sensor readings into a file sensor_log.txt. Each log entry consists of a type flag (1 for Temperature, 2 for Status, 3 for Count).
Write a C program to:
1. Define a tagged union structure SensorReading (with enum for type and union for float, char[20], and int).
2. Parse the formatted input file line-by-line using file functions (fopen, fscanf, fclose).
3. Store entries in a dynamically allocated array of SensorReading structures and output formatted logs based on their union payload type.

Input Format

A sequence of inputs where each line represents a record: <TypeFlag> <Value>.
The last line contains -1 indicating End-Of-File / stop reading.

Output Format

Print each record with its mapped tag: [TEMP] ..., [STATUS] ..., [COUNT] ...

Sample Input	Sample Output
1 36.5 2 OK 3 142 -1	[TEMP] 36.50 C [STATUS] OK [COUNT] 142

2 Python Solution

```
import os
import sys

TEMP, STATUS, COUNT = 1, 2, 3


class SensorReading:
    def __init__(self, tag, value):
        self.tag = tag      # type flag
        self.value = value  # float, str or int depending on tag


if os.path.exists("sensor_log.txt"):
    f = open("sensor_log.txt", "r")
    lines = f.readlines()
    f.close()
else:
    lines = sys.stdin.readlines()

readings = []
for line in lines:
    parts = line.split()
    if len(parts) == 0:
        continue
    flag = int(parts[0])
    if flag == -1:
        break
    if flag == TEMP:
        readings.append(SensorReading(TEMP, float(parts[1])))
    elif flag == STATUS:
```

```
31          readings.append(SensorReading(STATUS, " ".join(parts[1:])))
32      elif flag == COUNT:
33          readings.append(SensorReading(COUNT, int(parts[1])))
34
35  for r in readings:
36      if r.tag == TEMP:
37          print("[TEMP] %.2f C" % r.value)
38      elif r.tag == STATUS:
39          print("[STATUS] " + r.value)
40      else:
41          print("[COUNT] " + str(r.value))
```

3 Line-by-Line Explanation

Line	Code	Explanation
1	`import os`	os.path.exists checks whether a file exists.
2	`import sys`	sys.stdin is used when there is no file.
4	`TEMP, STATUS, COUNT = 1, 2, 3`	Named constants for the three type flags (like an enum in C).
7	`class SensorReading:`	Tagged union: a tag plus one value whose meaning depends on the tag.
8	`    def __init__(self, tag, value):`	Constructor that receives the tag and the value.
9	`        self.tag = tag      # type flag`	Store the type flag (1, 2 or 3).
10	`        self.value = value  # float, str or int depending on tag`	Store the payload: float, text or int depending on the tag.
13	`if os.path.exists("sensor_log.txt"):`	If the file sensor_log.txt exists...
14	`    f = open("sensor_log.txt", "r")`	...open it for reading (like fopen).
15	`    lines = f.readlines()`	Read all its lines into a list.
16	`    f.close()`	Close the file (like fclose).
17	`else:`	Otherwise...
18	`    lines = sys.stdin.readlines()`	...read the lines typed on the keyboard.
20	`readings = []`	Dynamic list that stores all SensorReading objects.
21	`for line in lines:`	Process the log one line at a time.
22	`    parts = line.split()`	Split the line into the flag and the value.
23	`    if len(parts) == 0:`	If the line is empty...
24	`        continue`	...skip it.
25	`    flag = int(parts[0])`	The first token is the type flag.
26	`    if flag == -1:`	-1 marks the end of the log.
27	`        break`	Stop reading.
28	`    if flag == TEMP:`	Flag 1: temperature.
29	`        readings.append(SensorReading(TEMP, float(parts[1])))`	Store the value as a decimal number.
30	`    elif flag == STATUS:`	Flag 2: status text.
31	`        readings.append(SensorReading(STATUS, " ".join(parts[1:])))`	Store the text.
32	`    elif flag == COUNT:`	Flag 3: count.
33	`        readings.append(SensorReading(COUNT, int(parts[1])))`	Store the value as an integer.
35	`for r in readings:`	Print every stored reading.
36	`    if r.tag == TEMP:`	Temperature reading.
37	`        print("[TEMP] %.2f C" % r.value)`	Print with 2 decimals and the unit C.

Line	Code	Explanation
38	`elif r.tag == STATUS:`	Status reading.
39	`print("[STATUS] " + r.value)`	Print the tag and the status text.
40	`else:`	Anything else is a count reading.
41	`print("[COUNT] " + str(r.value))`	Print the tag and the count.

4 Dry Run (Sample Test Case)

Input lines: 1 36.5 / 2 OK / 3 142 / -1. The program reads the lines one by one.

Line	flag	Action	readings list after
1 36.5	1	Store SensorReading(TEMP, 36.5)	[TEMP 36.5]
2 OK	2	Store SensorReading(STATUS, 'OK')	[TEMP 36.5, STATUS OK]
3 142	3	Store SensorReading(COUNT, 142)	[TEMP 36.5, STATUS OK, COUNT 142]
-1	-1	break: stop reading	unchanged

Result

```
Printing the three readings gives:
[TEMP] 36.50 C
[STATUS] OK
[COUNT] 142
```

Session 5: Linked Lists in C

Question 5.1

Reverse Nodes in k-Group

Marks: 10 | Bloom's Taxonomy Level: L3 (Apply)

1 Problem

Given a singly linked list, reverse the nodes of the linked list k at a time and return its modified list. If the number of nodes is not a multiple of k, then left-out nodes in the end should remain as it is.

Constraints / Requirements

- Time Complexity: O(N)
- Space Complexity: O(1) auxiliary space (in-place modification).
- Only modify the pointers, do not swap data values.

Input Format

First line: Integer N (number of nodes) and Integer K.
Second line: N space-separated integers representing the linked list.

Output Format

Space-separated integers representing the modified linked list.

Sample Input	Sample Output
5 2 1 2 3 4 5	2 1 4 3 5

2 Python Solution

```
import sys


class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


def reverse_k_group(head, k):
    dummy = Node(0)
    dummy.next = head
    prev = dummy
    while True:
        end = prev
        for _ in range(k):
            if end is None:
                break
            end = end.next
        if end is None:
            break

        start = prev.next
        after = end.next
        p = after
        cur = start
        while cur is not after:
            nxt = cur.next
            cur.next = p
            p = cur
            cur = nxt
```

```
32          prev.next = end
33          prev = start
34      return dummy.next
35
36
37  data = sys.stdin.read().split()
38  n = int(data[0])
39  k = int(data[1])
40  head = None
41  tail = None
42  for i in range(n):
43      node = Node(int(data[2 + i]))
44      if head is None:
45          head = node
46          tail = node
47      else:
48          tail.next = node
49          tail = node
50
51  head = reverse_k_group(head, k)
52
53  out = []
54  cur = head
55  while cur is not None:
56      out.append(str(cur.data))
57      cur = cur.next
58  print(" ".join(out))
```

3 Line-by-Line Explanation

Line	Code	Explanation
1	`import sys`	Used to read all the input at once.
4	`class Node:`	One node of the singly linked list.
5	`    def __init__(self, data):`	Constructor.
6	`        self.data = data`	The value stored in the node.
7	`        self.next = None`	Pointer to the next node (None means the end).
10	`def reverse_k_group(head, k):`	Reverses the list k nodes at a time.
11	`    dummy = Node(0)`	Dummy node placed before the head, to make edge cases simple.
12	`    dummy.next = head`	Link the dummy node to the real head.
13	`    prev = dummy`	prev is the node just before the group being reversed.
14	`    while True:`	Process group after group.
15	`        end = prev`	Start at prev and walk k steps to find the last node of the group.
16	`        for _ in range(k):`	Repeat k times.
17	`            if end is None:`	Ran out of nodes...
18	`                break`	...stop walking.
19	`            end = end.next`	Move one node forward.
20	`        if end is None:`	Fewer than k nodes remain...
21	`            break`	...leave them as they are and finish.
23	`        start = prev.next`	First node of the group.
24	`        after = end.next`	First node after the group.
25	`        p = after`	The reversed part will point to the node after the group.
26	`        cur = start`	cur walks through the group.
27	`        while cur is not after:`	Reverse the pointers of all k nodes.

Line	Code	Explanation
28	`            nxt = cur.next`	Save the next node before changing the pointer.
29	`            cur.next = p`	Point the current node backwards.
30	`            p = cur`	p moves forward to the current node.
31	`            cur = nxt`	Move to the saved next node.
32	`        prev.next = end`	The node before the group now points to the new first node (old last).
33	`        prev = start`	The old first node is now the group's tail; it becomes the new prev.
34	`    return dummy.next`	Return the real head of the modified list.
37	`data = sys.stdin.read().split()`	Read all input tokens.
38	`n = int(data[0])`	Number of nodes.
39	`k = int(data[1])`	Group size.
40	`head = None`	Head of the list, empty at first.
41	`tail = None`	Tail of the list, empty at first.
42	`for i in range(n):`	Create N nodes.
43	`    node = Node(int(data[2 + i]))`	Make a node with the next value.
44	`    if head is None:`	If the list is empty...
45	`        head = node`	...this node is the head...
46	`        tail = node`	...and also the tail.
47	`    else:`	Otherwise...
48	`        tail.next = node`	...attach the node after the tail...
49	`        tail = node`	...and make it the new tail.
51	`head = reverse_k_group(head, k)`	Reverse the list in groups of k.
53	`out = []`	Collects the values as text.
54	`cur = head`	Start at the head.
55	`while cur is not None:`	Walk through the list.
56	`    out.append(str(cur.data))`	Add the value to the output list.
57	`    cur = cur.next`	Move to the next node.
58	`print(" ".join(out))`	Print the values separated by spaces.

4 Dry Run (Sample Test Case)

Input: N = 5, k = 2, list 1 -> 2 -> 3 -> 4 -> 5. A dummy node sits before 1 and prev starts at the dummy.

Round	Group found	Pointer work	List after
1	1, 2 (start = 1, end = 2, after = 3)	cur = 1: 1.next = 3. cur = 2: 2.next = 1. Then dummy.next = 2, prev = node 1	2 -> 1 -> 3 -> 4 -> 5
2	3, 4 (start = 3, end = 4, after = 5)	cur = 3: 3.next = 5. cur = 4: 4.next = 3. Then node 1.next = 4, prev = node 3	2 -> 1 -> 4 -> 3 -> 5
3	Only node 5 is left (fewer than k = 2 nodes)	end becomes None, so break	2 -> 1 -> 4 -> 3 -> 5

Result

The leftover node 5 stays as it is. Output: 2 1 4 3 5

Question 5.2

Rotate Doubly Linked List

Marks: 10 | Bloom's Taxonomy Level: L3 (Apply)

1 Problem

Given a doubly linked list, rotate the linked list counter-clockwise by P nodes. Here P is a given positive integer and is smaller than the count of nodes in linked list.

Constraints / Requirements

- Time Complexity: O(N)
- Space Complexity: O(1)
- Ensure the previous (prev) and next (next) pointers are properly updated for all affected nodes.

Input Format

First line: Integer N (number of nodes) and Integer P (rotation count).
Second line: N space-separated integers.

Output Format

Space-separated integers of the rotated doubly linked list.

Sample Input	Sample Output
6 2 1 2 3 4 5 6	3 4 5 6 1 2

2 Python Solution

```
import sys


class Node:
    def __init__(self, data):
        self.data = data
        self.prev = None
        self.next = None


data = sys.stdin.read().split()
n = int(data[0])
p = int(data[1])

head = None
tail = None
for i in range(n):
    node = Node(int(data[2 + i]))
    if head is None:
        head = node
        tail = node
    else:
        tail.next = node
        node.prev = tail
        tail = node

p = p % n
if p != 0:
    nth = head
    for _ in range(p - 1):
        nth = nth.next
    new_head = nth.next
    tail.next = head
    head.prev = tail
    new_head.prev = None
```

```
    nth.next = None
    head = new_head

out = []
cur = head
while cur is not None:
    out.append(str(cur.data))
    cur = cur.next
print(" ".join(out))
```

3 Line-by-Line Explanation

Line	Code	Explanation
1	`import sys`	Used to read all the input at once.
4	`class Node:`	One node of the doubly linked list.
5	`    def __init__(self, data):`	Constructor.
6	`        self.data = data`	The value stored in the node.
7	`        self.prev = None`	Pointer to the previous node.
8	`        self.next = None`	Pointer to the next node.
11	`data = sys.stdin.read().split()`	Read all input tokens.
12	`n = int(data[0])`	Number of nodes.
13	`p = int(data[1])`	Number of nodes to rotate by.
15	`head = None`	Head of the list, empty at first.
16	`tail = None`	Tail of the list, empty at first.
17	`for i in range(n):`	Create N nodes.
18	`    node = Node(int(data[2 + i]))`	Make a node with the next value.
19	`    if head is None:`	If the list is empty...
20	`        head = node`	...this node is the head...
21	`        tail = node`	...and also the tail.
22	`    else:`	Otherwise...
23	`        tail.next = node`	...the old tail points forward to the new node...
24	`        node.prev = tail`	...the new node points back to the old tail...
25	`        tail = node`	...and becomes the new tail.
27	`p = p % n`	A rotation by n or more is the same as rotating by p mod n.
28	`if p != 0:`	Rotate only when needed.
29	`    nth = head`	Start at the head to find the p-th node.
30	`    for _ in range(p - 1):`	Move p - 1 steps forward.
31	`        nth = nth.next`	Step to the next node.
32	`    new_head = nth.next`	The node after the p-th node becomes the new head.
33	`    tail.next = head`	Old tail points forward to the old head (makes the list circular).
34	`    head.prev = tail`	Old head points back to the old tail.
35	`    new_head.prev = None`	The new head has no previous node.
36	`    nth.next = None`	The p-th node becomes the new tail, so it has no next node.
37	`    head = new_head`	Update the head.
39	`out = []`	Collects the values as text.
40	`cur = head`	Start at the head.

Line	Code	Explanation
41	`while cur is not None:`	Walk through the list.
42	`    out.append(str(cur.data))`	Add the value to the output list.
43	`    cur = cur.next`	Move to the next node.
44	`print(" ".join(out))`	Print the values separated by spaces.

4 Dry Run (Sample Test Case)

Input: N = 6, P = 2, list 1 <-> 2 <-> 3 <-> 4 <-> 5 <-> 6. p % n = 2.

Step	Action	Result
1	Find nth: start at head (1) and move p - 1 = 1 step	nth = node 2
2	new_head = nth.next	new_head = node 3
3	tail.next = head and head.prev = tail	node 6 <-> node 1 (list is circular)
4	new_head.prev = None and nth.next = None	list is cut between 2 and 3
5	head = new_head	head = node 3

Result

Reading from the new head: 3 4 5 6 1 2. Output: 3 4 5 6 1 2

Session 6: STL - Stacks, Queues & Priority Queues

Question 6.1

Next Greater Element

Marks: 10 | Bloom's Taxonomy Level: L3 (Apply)

1 Problem

Given an array of integers, find the Next Greater Element (NGE) for every element. The Next greater Element for an element x is the first greater element on the right side of x in the array. Elements for which no greater element exist, consider next greater element as -1.

Constraints / Requirements

- Must be solved using a Stack data structure.
- Time Complexity: O(N)
- Space Complexity: O(N)

Input Format

First line: Integer N.
Second line: N space-separated integers.

Output Format

N space-separated integers representing the NGE for each element.

Sample Input	Sample Output
4 4 5 2 25	5 25 25 -1

2 Python Solution

```
 1  import sys
 2
 3  data = sys.stdin.read().split()
 4  n = int(data[0])
 5  a = [int(data[i + 1]) for i in range(n)]
 6
 7  res = [-1] * n
 8  stack = []   # stores indices
 9  for i in range(n):
10      while stack and a[stack[-1]] < a[i]:
11          res[stack.pop()] = a[i]
12      stack.append(i)
13
14  print(" ".join(str(x) for x in res))
```

3 Line-by-Line Explanation

Line	Code	Explanation
1	`import sys`	Used to read all the input at once.
3	`data = sys.stdin.read().split()`	Read all input tokens.
4	`n = int(data[0])`	Number of elements.
5	`a = [int(data[i + 1]) for i in range(n)]`	Build the list of N integers.
7	`res = [-1] * n`	Answer list, filled with -1 (meaning: no greater element yet).

Line	Code	Explanation
8	`stack = []  # stores indices`	Stack of indices whose next greater element is not found yet.
9	`for i in range(n):`	Go through the array from left to right.
10	`    while stack and a[stack[-1]] < a[i]:`	While the stack is not empty and the top element is smaller than a[i]...
11	`        res[stack.pop()] = a[i]`	...a[i] is the next greater element of the top, so record it and pop.
12	`    stack.append(i)`	Push the current index; its answer is not known yet.
14	`print(" ".join(str(x) for x in res))`	Print the answers separated by spaces.

4 Dry Run (Sample Test Case)

Input: a = [4, 5, 2, 25]. res = [-1, -1, -1, -1]. The stack stores indices.

i	a[i]	Pops (stack top < a[i])	Stack after push	res after
0	4	none (stack empty)	[0]	[-1, -1, -1, -1]
1	5	4 < 5: pop index 0, res[0] = 5	[1]	[5, -1, -1, -1]
2	2	5 < 2 is false: no pop	[1, 2]	[5, -1, -1, -1]
3	25	2 < 25: pop index 2, res[2] = 25; 5 < 25: pop index 1, res[1] = 25	[3]	[5, 25, 25, -1]

Result

Index 3 stays on the stack, so its answer remains -1. Output: 5 25 25 -1

Question 6.2

Minimum Cost of Ropes

Marks: 10 | Bloom's Taxonomy Level: L3 (Apply)

1 Problem

There are given N ropes of different lengths, we need to connect these ropes into one rope. The cost to connect two ropes is equal to sum of their lengths. The task is to connect the ropes with minimum cost.

Constraints / Requirements

- Must be solved using a Priority Queue (Min-Heap).
- Time Complexity: O(N log N)
- Space Complexity: O(N)

Input Format

First line: Integer N.
Second line: N space-separated integers representing rope lengths.

Output Format

A single integer representing the minimum cost.

Sample Input	Sample Output
4 4 3 2 6	29

2 Python Solution

```
 1  import heapq
 2  import sys
 3
 4  data = sys.stdin.read().split()
 5  n = int(data[0])
 6  heap = [int(data[i + 1]) for i in range(n)]
 7  heapq.heapify(heap)
 8
 9  cost = 0
10  while len(heap) > 1:
11      a = heapq.heappop(heap)
12      b = heapq.heappop(heap)
13      cost += a + b
14      heapq.heappush(heap, a + b)
15
16  print(cost)
```

3 Line-by-Line Explanation

Line	Code	Explanation
1	`import heapq`	heapq provides a min-heap (priority queue).
2	`import sys`	Used to read all the input at once.
4	`data = sys.stdin.read().split()`	Read all input tokens.
5	`n = int(data[0])`	Number of ropes.
6	`heap = [int(data[i + 1]) for i in range(n)]`	List of rope lengths.
7	`heapq.heapify(heap)`	Turn the list into a min-heap.
9	`cost = 0`	Total cost so far.
10	`while len(heap) > 1:`	Continue until only one rope is left.

Line	Code	Explanation
11	`a = heapq.heappop(heap)`	Take the shortest rope.
12	`b = heapq.heappop(heap)`	Take the next shortest rope.
13	`cost += a + b`	Joining them costs the sum of their lengths.
14	`heapq.heappush(heap, a + b)`	Put the joined rope back in the heap.
16	`print(cost)`	Print the minimum total cost.

4 Dry Run (Sample Test Case)

Input: ropes = 4 3 2 6. The min-heap always gives the two shortest ropes.

Step	Heap before	Pop a, b	Cost of joining	Total cost	Heap after
1	[2, 3, 4, 6]	2, 3	2 + 3 = 5	5	[4, 5, 6]
2	[4, 5, 6]	4, 5	4 + 5 = 9	14	[6, 9]
3	[6, 9]	6, 9	6 + 9 = 15	29	[15]

Result

Only one rope is left, so the loop ends. Output: 29

Session 7: Linked Lists, Binary Trees, Stacks & Queues

Question 7.1

Zigzag Level Order Traversal

Marks: 10 | Bloom's Taxonomy Level: L3 (Apply)

1 Problem

Given the root of a binary tree, return the zigzag level order traversal of its nodes' values. (i.e., from left to right, then right to left for the next level and alternate between).

Constraints / Requirements

- Implement using a Queue/Deque or two Stacks.
- Time Complexity: O(N)
- Space Complexity: O(N)

Input Format

A single line containing the level-order representation of the tree (use -1 for NULL nodes).

Output Format

Space-separated integers representing the zigzag traversal.

Sample Input	Sample Output
`3 9 20 -1 -1 15 7`	`3 20 9 15 7`

2 Python Solution

```
import sys
from collections import deque


class Node:
    def __init__(self, val):
        self.val = val
        self.left = None
        self.right = None


vals = list(map(int, sys.stdin.read().split()))

if len(vals) == 0 or vals[0] == -1:
    print()
else:
    root = Node(vals[0])
    q = deque([root])
    i = 1
    while q and i < len(vals):
        cur = q.popleft()
        if i < len(vals):
            v = vals[i]
            i += 1
            if v != -1:
                cur.left = Node(v)
                q.append(cur.left)
        if i < len(vals):
            v = vals[i]
            i += 1
            if v != -1:
                cur.right = Node(v)
                q.append(cur.right)

```

```
    result = []
    q = deque([root])
    left_to_right = True
    while q:
        level = []
        for _ in range(len(q)):
            cur = q.popleft()
            level.append(cur.val)
            if cur.left is not None:
                q.append(cur.left)
            if cur.right is not None:
                q.append(cur.right)
        if not left_to_right:
            level.reverse()
        result.extend(level)
        left_to_right = not left_to_right

    print(" ".join(str(x) for x in result))
```

3 Line-by-Line Explanation

Line	Code	Explanation
1	`import sys`	Used to read all the input at once.
2	`from collections import deque`	deque is a fast queue.
5	`class Node:`	One node of the binary tree.
6	`    def __init__(self, val):`	Constructor.
7	`        self.val = val`	The value stored in the node.
8	`        self.left = None`	Left child.
9	`        self.right = None`	Right child.
12	`vals = list(map(int, sys.stdin.read().split()))`	Read the level-order values (-1 means NULL).
14	`if len(vals) == 0 or vals[0] == -1:`	If the tree is empty...
15	`    print()`	...print an empty line.
16	`else:`	Otherwise build the tree and traverse it.
17	`    root = Node(vals[0])`	The first value is the root.
18	`    q = deque([root])`	Queue of nodes whose children are still to be read.
19	`    i = 1`	Index of the next value to read.
20	`    while q and i < len(vals):`	Continue while there are nodes waiting and values left.
21	`        cur = q.popleft()`	Take the next node from the queue.
22	`        if i < len(vals):`	If a value is available for the left child...
23	`            v = vals[i]`	...read it...
24	`            i += 1`	...and move on.
25	`            if v != -1:`	If it is not NULL...
26	`                cur.left = Node(v)`	...create the left child...
27	`                q.append(cur.left)`	...and queue it.
28	`        if i < len(vals):`	If a value is available for the right child...
29	`            v = vals[i]`	...read it...
30	`            i += 1`	...and move on.
31	`            if v != -1:`	If it is not NULL...
32	`                cur.right = Node(v)`	...create the right child...
33	`                q.append(cur.right)`	...and queue it.
35	`    result = []`	Final zigzag order.

Line	Code	Explanation
36	`    q = deque([root])`	Restart the queue with the root for the traversal.
37	`    left_to_right = True`	The first level goes left to right.
38	`    while q:`	Process the tree level by level.
39	`        level = []`	Values of the current level.
40	`        for _ in range(len(q)):`	The queue holds exactly one level right now.
41	`            cur = q.popleft()`	Take the next node.
42	`            level.append(cur.val)`	Record its value.
43	`            if cur.left is not None:`	If there is a left child...
44	`                q.append(cur.left)`	...queue it for the next level.
45	`            if cur.right is not None:`	If there is a right child...
46	`                q.append(cur.right)`	...queue it for the next level.
47	`        if not left_to_right:`	On right-to-left levels...
48	`            level.reverse()`	...reverse the level.
49	`        result.extend(level)`	Add the level to the result.
50	`        left_to_right = not left_to_right`	Flip the direction for the next level.
52	`    print(" ".join(str(x) for x in result))`	Print the zigzag order separated by spaces.

4 Dry Run (Sample Test Case)

Input: 3 9 20 -1 -1 15 7. Building the tree: 3 is the root; its children are 9 and 20; 9 has no children (-1, -1); 20 has children 15 and 7. Then the traversal runs level by level.

Level	Nodes in queue (left to right)	Direction	Added to result	Result so far
0	3	Left to right	3	3
1	9, 20	Right to left (reversed)	20, 9	3 20 9
2	15, 7	Left to right	15, 7	3 20 9 15 7

Result

```
Output: 3 20 9 15 7
```

Question 7.2

Implement Queue using Two Stacks

Marks: 10 | Bloom's Taxonomy Level: L3 (Apply)

1 Problem

Implement a Queue data structure using only two Stacks. The implemented queue should support standard operations: enqueue (push to back), dequeue (remove from front), and peek (view front element).

Constraints / Requirements

- Use standard stack operations only (push, pop, top, empty).
- Amortized Time Complexity for Dequeue: O(1).

Input Format

First line: Integer Q (number of queries).
Next Q lines: Query type 1 x (enqueue x), type 2 (dequeue and print), type 3 (peek and print).

Output Format

Outputs for queries of type 2 and 3 on new lines.

Sample Input	Sample Output
5 1 10 1 20 2 1 30 3	10 20

2 Python Solution

```
import sys

data = sys.stdin.read().split()
q = int(data[0])
idx = 1

in_stack = []
out_stack = []


def shift():
    if len(out_stack) == 0:
        while len(in_stack) > 0:
            out_stack.append(in_stack.pop())


for _ in range(q):
    t = int(data[idx])
    idx += 1
    if t == 1:
        in_stack.append(int(data[idx]))
        idx += 1
    elif t == 2:
        shift()
        if len(out_stack) > 0:
            print(out_stack.pop())
    else:
        shift()
        if len(out_stack) > 0:
            print(out_stack[-1])
```

3 Line-by-Line Explanation

Line	Code	Explanation
1	`import sys`	Used to read all the input at once.
3	`data = sys.stdin.read().split()`	Read all input tokens.
4	`q = int(data[0])`	Number of queries.
5	`idx = 1`	Index of the next unread token.
7	`in_stack = []`	Stack that receives new elements (enqueue).
8	`out_stack = []`	Stack that gives elements out in queue order (dequeue/peek).
11	`def shift():`	Moves elements from in_stack to out_stack when needed.
12	`    if len(out_stack) == 0:`	Only when out_stack is empty...
13	`        while len(in_stack) > 0:`	...move everything...
14	`            out_stack.append(in_stack.pop())`	...which reverses the order, so the oldest element ends on top.
17	`for _ in range(q):`	Process each query.
18	`    t = int(data[idx])`	The query type.
19	`    idx += 1`	Move to the next token.
20	`    if t == 1:`	Type 1: enqueue.
21	`        in_stack.append(int(data[idx]))`	Push the value on in_stack.
22	`        idx += 1`	Skip over the value just read.
23	`    elif t == 2:`	Type 2: dequeue and print.
24	`        shift()`	Make sure out_stack has the front element.
25	`        if len(out_stack) > 0:`	Only if the queue is not empty...
26	`            print(out_stack.pop())`	...remove and print the front element.
27	`    else:`	Type 3: peek and print.
28	`        shift()`	Make sure out_stack has the front element.
29	`        if len(out_stack) > 0:`	Only if the queue is not empty...
30	`            print(out_stack[-1])`	...print the front element without removing it.

4 Dry Run (Sample Test Case)

Input: 5 queries. in_stack receives new elements; out_stack gives the front of the queue.

Query	in_stack	out_stack	Printed
1 10 (enqueue)	[10]	[]	-
1 20 (enqueue)	[10, 20]	[]	-
2 (dequeue)	[]	shift moves 20 then 10: [20, 10]; pop gives 10, leaving [20]	10
1 30 (enqueue)	[30]	[20]	-
3 (peek)	[30]	[20] is not empty, so no shift; top is 20	20

Result

```
Output:
10
20
```

Session 8: String Matching

Question 8.1

Rabin-Karp Pattern Matching

Marks: 10 | Bloom's Taxonomy Level: L3 (Apply)

1 Problem

Given a text string T and a pattern string P, write a program implementing the Rabin-Karp algorithm to find all starting indices of the pattern in the text.

Constraints / Requirements

- Use a prime modulus to calculate the rolling hash.
- Handle hash collisions properly by verifying string characters.
- Time Complexity: Average O(N+M), Worst O(N*M).

Input Format

First line: Text string T.
Second line: Pattern string P.

Output Format

Space-separated starting indices (0-indexed). Print -1 if no match.

Sample Input	Sample Output
AABAACAADAABAABA AABA	0 9 12

2 Python Solution

```
text = input().strip("\r\n")
pat = input().strip("\r\n")
n = len(text)
m = len(pat)
d = 256
q = 1000000007

result = []
if 0 < m <= n:
    h = 1
    for _ in range(m - 1):
        h = (h * d) % q

    ph = 0
    th = 0
    for i in range(m):
        ph = (ph * d + ord(pat[i])) % q
        th = (th * d + ord(text[i])) % q

    for i in range(n - m + 1):
        if ph == th:
            match = True
            for j in range(m):
                if text[i + j] != pat[j]:
                    match = False
                    break
            if match:
                result.append(i)
        if i + m < n:
            th = (th - ord(text[i]) * h) % q
            th = (th * d + ord(text[i + m])) % q
```

```
32
33 if len(result) == 0:
34     print(-1)
35 else:
36     print(" ".join(str(x) for x in result))
```

3 Line-by-Line Explanation

Line	Code	Explanation
1	`text = input().strip("\r\n")`	Read the text line.
2	`pat = input().strip("\r\n")`	Read the pattern line.
3	`n = len(text)`	Length of the text.
4	`m = len(pat)`	Length of the pattern.
5	`d = 256`	Base of the rolling hash (number of possible characters).
6	`q = 1000000007`	Prime modulus used to keep the hash values small.
8	`result = []`	Starting indices of matches.
9	`if 0 < m <= n:`	Search only when the pattern is not empty and not longer than the text.
10	`    h = 1`	h will become d^(m-1) mod q.
11	`    for _ in range(m - 1):`	Repeat m - 1 times.
12	`        h = (h * d) % q`	Multiply by d each time.
14	`    ph = 0`	Hash of the pattern.
15	`    th = 0`	Hash of the current text window.
16	`    for i in range(m):`	Hash the pattern and the first window together.
17	`        ph = (ph * d + ord(pat[i])) % q`	Add the next pattern character to the pattern hash.
18	`        th = (th * d + ord(text[i])) % q`	Add the next text character to the window hash.
20	`    for i in range(n - m + 1):`	Slide the window over every possible start position.
21	`        if ph == th:`	If the hashes are equal, the window may match...
22	`            match = True`	...assume a match...
23	`            for j in range(m):`	...and verify character by character (handles collisions).
24	`                if text[i + j] != pat[j]:`	A character differs...
25	`                    match = False`	...so it was only a hash collision...
26	`                    break`	...stop checking.
27	`            if match:`	All characters matched...
28	`                result.append(i)`	...so record the start index.
29	`        if i + m < n:`	If there is another window to the right...
30	`            th = (th - ord(text[i]) * h) % q`	...remove the leading character from the hash...
31	`            th = (th * d + ord(text[i + m])) % q`	...and add the next character (rolling hash).
33	`if len(result) == 0:`	No match was found...
34	`    print(-1)`	...print -1.
35	`else:`	Otherwise...
36	`    print(" ".join(str(x) for x in result))`	...print all start indices separated by spaces.

4 Dry Run (Sample Test Case)

Input: text = AABAACAADAABAABA (length 16), pattern = AABA (length 4). Pattern hash ph = 94795834. The window hash th is rolled forward one character at a time (remove the first character, add the next). Characters are compared only when the hashes are equal.

Start i	Window	Window hash th	th == ph ?	Characters verified
0	AABA	94795834	Yes	Yes, match
1	ABAA	94861114	No	-
2	BAAC	111572796	No	-
3	AACA	94796090	No	-
4	ACAA	94926650	No	-
5	CAAD	128350013	No	-
6	AADA	94796346	No	-
7	ADAA	94992186	No	-
8	DAAB	145127227	No	-
9	AABA	94795834	Yes	Yes, match
10	ABAA	94861114	No	-
11	BAAB	111572795	No	-
12	AABA	94795834	Yes	Yes, match

Result

Matches found at indices 0, 9, 12. Output: 0 9 12

Question 8.2

Longest Palindromic Substring

Marks: 10 | Bloom's Taxonomy Level: L3 (Apply)

1 Problem

Given a string S, find the longest palindromic substring in S. A palindrome is a string that reads the same backwards as forwards.

Constraints / Requirements

- Time Complexity: O(N^2) using expand around center approach or dynamic programming.
- Space Complexity: O(1) or O(N^2).

Input Format

A single string S (length <= 1000).

Output Format

The longest palindromic substring.

Sample Input	Sample Output
babad	bab

2 Python Solution

```
s = input().strip("\r\n")
n = len(s)
best_start = 0
best_len = 0

for c in range(n):
    for extra in range(2):          # 0 = odd length, 1 = even length
        l = c
        r = c + extra
        while l >= 0 and r < n and s[l] == s[r]:
            l -= 1
            r += 1
        length = r - l - 1
        if length > best_len:
            best_len = length
            best_start = l + 1

print(s[best_start:best_start + best_len])
```

3 Line-by-Line Explanation

Line	Code	Explanation
1	`s = input().strip("\r\n")`	Read the string.
2	`n = len(s)`	Length of the string.
3	`best_start = 0`	Start index of the longest palindrome found so far.
4	`best_len = 0`	Length of the longest palindrome found so far.
6	`for c in range(n):`	Treat every position as a possible centre.
7	`for extra in range(2):  # 0 = odd length, 1 = even length`	extra = 0 tests an odd palindrome (one centre); extra = 1 tests an even palindrome (two centres).
8	`l = c`	Left pointer starts at the centre.
9	`r = c + extra`	Right pointer starts at the centre (odd) or the next position (even).

Line	Code	Explanation
10	`while l >= 0 and r < n and s[l] == s[r]:`	Expand outwards while the characters match.
11	`l -= 1`	Move the left pointer left.
12	`r += 1`	Move the right pointer right.
13	`length = r - l - 1`	Length of the palindrome found (the pointers stopped one step too far).
14	`if length > best_len:`	If it is longer than the best so far...
15	`best_len = length`	...save the new length...
16	`best_start = l + 1`	...and its start index.
18	`print(s[best_start:best_start + best_len])`	Print the longest palindromic substring.

4 Dry Run (Sample Test Case)

Input: babad (indices 0 b, 1 a, 2 b, 3 a, 4 d). For each centre we expand while characters match. length = r - l - 1 after the loop.

Centre c	Type	Final l, r	length	Palindrome	Best so far
0	odd	-1, 1	1	b	len 1, start 0
0	even	0, 1	0	(a, b differ)	unchanged
1	odd	-1, 3	3	bab	len 3, start 0
1	even	1, 2	0	(a, b differ)	unchanged
2	odd	0, 4	3	aba	3 is not > 3, unchanged
2	even	2, 3	0	(b, a differ)	unchanged
3	odd	2, 4	1	a	unchanged
3	even	3, 4	0	(a, d differ)	unchanged
4	odd	3, 5	1	d	unchanged
4	even	4, 5	0	(end of string)	unchanged

Result

`best_start = 0 and best_len = 3, so s[0:3] = bab. Output: bab`

`(Because 'bab' is found first and 'aba' is not longer, 'bab' is printed, matching the question bank.)`

Session 9: Performance Analysis of Sorting and Searching

Question 9.1

Search in Rotated Sorted Array

Marks: 10 | Bloom's Taxonomy Level: L3 (Apply)

1 Problem

There is an integer array nums sorted in ascending order (with distinct values). Prior to being passed to your function, nums is possibly rotated at an unknown pivot index k. Given the array nums after the possible rotation and an integer target, return the index of target if it is in nums, or -1 if it is not in nums.

Constraints / Requirements

- You must write an algorithm with O(log N) runtime complexity (Binary Search).

Input Format

First line: Integer N.
Second line: N space-separated integers.
Third line: Integer Target.

Output Format

Integer index of the target, or -1.

Sample Input	Sample Output
7 4 5 6 7 0 1 2 0	4

2 Python Solution

```
import sys

data = sys.stdin.read().split()
n = int(data[0])
a = [int(data[i + 1]) for i in range(n)]
target = int(data[n + 1])

lo = 0
hi = n - 1
ans = -1
while lo <= hi:
    mid = (lo + hi) // 2
    if a[mid] == target:
        ans = mid
        break
    if a[lo] <= a[mid]:
        if a[lo] <= target < a[mid]:
            hi = mid - 1
        else:
            lo = mid + 1
    else:
        if a[mid] < target <= a[hi]:
            lo = mid + 1
        else:
            hi = mid - 1

print(ans)
```

3 Line-by-Line Explanation

Line	Code	Explanation
1	`import sys`	Used to read all the input at once.
3	`data = sys.stdin.read().split()`	Read all input tokens.
4	`n = int(data[0])`	Number of elements.
5	`a = [int(data[i + 1]) for i in range(n)]`	The rotated array.
6	`target = int(data[n + 1])`	The value to search for.
8	`lo = 0`	Left end of the search range.
9	`hi = n - 1`	Right end of the search range.
10	`ans = -1`	Answer, -1 until the target is found.
11	`while lo <= hi:`	Binary search while the range is not empty.
12	`    mid = (lo + hi) // 2`	Middle index.
13	`    if a[mid] == target:`	Found the target...
14	`        ans = mid`	...save its index...
15	`        break`	...and stop.
16	`    if a[lo] <= a[mid]:`	The left half lo..mid is sorted.
17	`        if a[lo] <= target < a[mid]:`	If the target lies inside the sorted left half...
18	`            hi = mid - 1`	...search the left half.
19	`        else:`	Otherwise...
20	`            lo = mid + 1`	...search the right half.
21	`    else:`	Otherwise the right half mid..hi is sorted.
22	`        if a[mid] < target <= a[hi]:`	If the target lies inside the sorted right half...
23	`            lo = mid + 1`	...search the right half.
24	`        else:`	Otherwise...
25	`            hi = mid - 1`	...search the left half.
27	`print(ans)`	Print the index, or -1 if not found.

4 Dry Run (Sample Test Case)

Input: a = [4, 5, 6, 7, 0, 1, 2], target = 0. Indices 0 to 6.

lo	hi	mid	a[mid]	Which half is sorted?	Decision
0	6	3	7	a[lo] = 4 <= 7: left half sorted	4 <= 0 < 7 is false, so lo = 4
4	6	5	1	a[lo] = 0 <= 1: left half sorted	0 <= 0 < 1 is true, so hi = 4
4	4	4	0	-	a[mid] == target: ans = 4

Result

```
Output: 4
```

Question 9.2

Inversion Count (Merge Sort)

Marks: 10 | Bloom's Taxonomy Level: L3 (Apply)

1 Problem

Given an array of integers. Find the Inversion Count in the array. Two array elements arr[i] and arr[j] form an inversion if arr[i] > arr[j] and i < j.

Constraints / Requirements

- Use a modified Merge Sort approach to count inversions during the merge step.
- Time Complexity: O(N log N)
- Space Complexity: O(N)

Input Format

First line: Integer N.
Second line: N space-separated integers.

Output Format

A single integer representing the inversion count.

Sample Input	Sample Output
5 2 4 1 3 5	3

2 Python Solution

```
import sys


def sort_and_count(a):
    if len(a) <= 1:
        return a, 0
    mid = len(a) // 2
    left, c1 = sort_and_count(a[:mid])
    right, c2 = sort_and_count(a[mid:])
    count = c1 + c2

    merged = []
    i = 0
    j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            merged.append(left[i])
            i += 1
        else:
            merged.append(right[j])
            j += 1
            count += len(left) - i
    while i < len(left):
        merged.append(left[i])
        i += 1
    while j < len(right):
        merged.append(right[j])
        j += 1
    return merged, count


data = sys.stdin.read().split()
n = int(data[0])
arr = [int(data[i + 1]) for i in range(n)]
_, total = sort_and_count(arr)
```

```
36  print(total)
```

3 Line-by-Line Explanation

Line	Code	Explanation
1	`import sys`	Used to read all the input at once.
4	`def sort_and_count(a):`	Merge sort that also returns the number of inversions.
5	`    if len(a) <= 1:`	A list of 0 or 1 element is already sorted...
6	`        return a, 0`	...and has 0 inversions.
7	`    mid = len(a) // 2`	Split point.
8	`    left, c1 = sort_and_count(a[:mid])`	Sort the left half and count its inversions.
9	`    right, c2 = sort_and_count(a[mid:])`	Sort the right half and count its inversions.
10	`    count = c1 + c2`	Inversions inside each half.
12	`    merged = []`	The merged sorted list.
13	`    i = 0`	Pointer into the left half.
14	`    j = 0`	Pointer into the right half.
15	`    while i < len(left) and j < len(right):`	Merge while both halves have elements.
16	`        if left[i] <= right[j]:`	The left element is not larger...
17	`            merged.append(left[i])`	...so take it (no inversion)...
18	`            i += 1`	...and advance the left pointer.
19	`        else:`	The right element is smaller...
20	`            merged.append(right[j])`	...so take it...
21	`            j += 1`	...and advance the right pointer.
22	`            count += len(left) - i`	Every remaining left element is bigger than it and comes before it, so each forms an inversion.
23	`    while i < len(left):`	Copy any leftover left elements.
24	`        merged.append(left[i])`	Add one.
25	`        i += 1`	Advance.
26	`    while j < len(right):`	Copy any leftover right elements.
27	`        merged.append(right[j])`	Add one.
28	`        j += 1`	Advance.
29	`    return merged, count`	Return the sorted list and the inversion count.
32	`data = sys.stdin.read().split()`	Read all input tokens.
33	`n = int(data[0])`	Number of elements.
34	`arr = [int(data[i + 1]) for i in range(n)]`	The array.
35	`_, total = sort_and_count(arr)`	Run the function; we only need the count (_ ignores the sorted list).
36	`print(total)`	Print the inversion count.

4 Dry Run (Sample Test Case)

Input: [2, 4, 1, 3, 5]. The array is split into halves, sorted and merged; inversions are counted whenever a right element is taken before remaining left elements.

Merge	Left	Right	What happens	Inversions added
A	[2]	[4]	2 <= 4: take 2, then 4	0
B	[3]	[5]	3 <= 5: take 3, then 5	0
C	[1]	[3, 5]	1 <= 3: take 1, then copy 3, 5	0
D (final)	[2, 4]	[1, 3, 5]	2 <= 1 false: take 1 (right), pairs (2,1) and (4,1)	len(left) - i = 2
	i = 0, j = 1		2 <= 3: take 2 (left)	0
	i = 1, j = 1		4 <= 3 false: take 3 (right), pair (4,3)	len(left) - i = 1
	i = 1, j = 2		4 <= 5: take 4, then copy 5	0

Result

Total inversions = 2 + 1 = 3 (the pairs (2,1), (4,1), (4,3)). Output: 3

Session 10: Number Theory

Question 10.1

Modular Exponentiation

Marks: 10 | Bloom's Taxonomy Level: L3 (Apply)

1 Problem

Given three numbers X, Y, and P, compute (X^Y) % P. This is known as modular exponentiation and is widely used in cryptography.

Constraints / Requirements

- Must be solved using fast exponentiation (Binary Exponentiation).
- Time Complexity: O(log Y)
- Space Complexity: O(1)

Input Format

Three space-separated integers X, Y, and P.

Output Format

A single integer result.

Sample Input	Sample Output
2 3 5	3

2 Python Solution

```
1  x, y, p = map(int, input().split())
2
3  result = 1 % p
4  x = x % p
5  while y > 0:
6      if y % 2 == 1:
7          result = (result * x) % p
8      x = (x * x) % p
9      y = y // 2
10
11 print(result)
```

3 Line-by-Line Explanation

Line	Code	Explanation
1	`x, y, p = map(int, input().split())`	Read X, Y and P from one line.
3	`result = 1 % p`	Start with 1 (taken mod p so that p = 1 also works).
4	`x = x % p`	Reduce the base modulo p.
5	`while y > 0:`	Process the exponent bit by bit.
6	`    if y % 2 == 1:`	If the current bit of y is 1...
7	`        result = (result * x) % p`	...multiply the result by the current base.
8	`    x = (x * x) % p`	Square the base (mod p).
9	`    y = y // 2`	Shift y right by one bit.
11	`print(result)`	Print (X^Y) % P.

4 Dry Run (Sample Test Case)

Input: X = 2, Y = 3, P = 5. result = 1 % 5 = 1 and x = 2 % 5 = 2. Y = 3 is 11 in binary.

Loop	y	y odd?	result after	x after (x*x % p)	y after
1	3	Yes	(1 * 2) % 5 = 2	(2 * 2) % 5 = 4	1
2	1	Yes	(2 * 4) % 5 = 3	(4 * 4) % 5 = 1	0

Result

```
y = 0 ends the loop. 2^3 = 8 and 8 % 5 = 3. Output: 3
```

Question 10.2

Prime Query using Sieve & Prefix Sums

Marks: 10 | Bloom's Taxonomy Level: L3 (Apply)

1 Problem

Given Q queries, each query containing two numbers L and R. Find the count of prime numbers in the range [L, R] inclusive.

Constraints / Requirements

- Precompute primes using the Sieve of Eratosthenes.
- Use a prefix sum array to answer each query in O(1) time.
- Time Complexity: O(N log log N + Q) where N = 10^6.

Input Format

First line: Integer Q (number of queries).
Next Q lines: Two space-separated integers L and R.

Output Format

Q lines, each containing the count of primes for that query.

Sample Input	Sample Output
2 1 10 10 20	4 4

2 Python Solution

```
 1  import sys
 2
 3  N = 1000000
 4  is_prime = [True] * (N + 1)
 5  is_prime[0] = False
 6  is_prime[1] = False
 7  for i in range(2, 1001):
 8      if is_prime[i]:
 9          for j in range(i * i, N + 1, i):
10              is_prime[j] = False
11
12  prefix = [0] * (N + 1)
13  for i in range(1, N + 1):
14      prefix[i] = prefix[i - 1] + (1 if is_prime[i] else 0)
15
16  data = sys.stdin.read().split()
17  q = int(data[0])
18  out = []
19  idx = 1
20  for _ in range(q):
21      l = int(data[idx])
22      r = int(data[idx + 1])
23      idx += 2
24      if l < 1:
25          l = 1
26      out.append(str(prefix[r] - prefix[l - 1]))
27  print("\n".join(out))
```

3 Line-by-Line Explanation

Line	Code	Explanation
1	`import sys`	Used to read all the input at once.

Line	Code	Explanation
3	`N = 1000000`	Upper limit of the sieve.
4	`is_prime = [True] * (N + 1)`	Assume every number is prime at first.
5	`is_prime[0] = False`	0 is not prime.
6	`is_prime[1] = False`	1 is not prime.
7	`for i in range(2, 1001):`	Only i up to 1000 is needed because 1000 * 1000 = N.
8	`    if is_prime[i]:`	If i is still marked prime...
9	`        for j in range(i * i, N + 1, i):`	...mark its multiples starting from i*i...
10	`            is_prime[j] = False`	...as not prime.
12	`prefix = [0] * (N + 1)`	prefix[i] will be the number of primes from 1 to i.
13	`for i in range(1, N + 1):`	Build the prefix sums.
14	`    prefix[i] = prefix[i - 1] + (1 if is_prime[i] else 0)`	Previous count plus 1 if i is prime.
16	`data = sys.stdin.read().split()`	Read all input tokens.
17	`q = int(data[0])`	Number of queries.
18	`out = []`	Collects the answers.
19	`idx = 1`	Index of the next unread token.
20	`for _ in range(q):`	Handle each query.
21	`    l = int(data[idx])`	Left end L.
22	`    r = int(data[idx + 1])`	Right end R.
23	`    idx += 2`	Move past the two numbers.
24	`    if l < 1:`	Protect against L = 0.
25	`        l = 1`	Primes start at 2, so start counting from 1.
26	`    out.append(str(prefix[r] - prefix[l - 1]))`	Primes in [L, R] = prefix[R] - prefix[L-1].
27	`print("\n".join(out))`	Print one answer per line.

4 Dry Run (Sample Test Case)

Sieve: starting at 2, mark multiples of each prime as not prime. The primes up to 20 are 2, 3, 5, 7, 11, 13, 17, 19. The table shows prefix[i] (count of primes from 1 to i) for the numbers needed.

prefix table (prefix[0] = 0)

i	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
prime?	N	Y	Y	N	Y	N	Y	N	N	N	Y	N	Y	N	N	N	Y	N	Y	N
prefix[i]	0	1	2	2	3	3	4	4	4	4	5	5	6	6	6	6	7	7	8	8

Answering the queries

Query	Formula	Calculation	Answer
1 10	prefix[10] - prefix[0]	4 - 0	4
10 20	prefix[20] - prefix[9]	8 - 4	4

Result

```
Output:
4
4
```